

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956

ENATHUR, KANCHIPURAM - 631 561 DEPARTMENT OF COMPUTER
SCIENCE & ENGINEERING



NAME :V.manoj varma

REG NO :11249A408

CLASS : S4

SUBJECT CODE : BCSD244A12

SUBJECT NAME :DATA SCIENCE TOOLKIT LAB

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956

ENATHUR, KANCHIPURAM - 631 561 DEPARTMENT OF COMPUTER
SCIENCE & ENGINEERING



BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by

Mr/Ms. _____ V.manoj varma _____

with Reg. No 11249A408 done by of II-B.E.(CSE) in –

DATA SCIENCE TOOLKIT LAB for Lab during the academic year 2025 2026.

Station : ENATHUR Date :

Staff-in-charge
Department

Head of the

Submitted for the practical examination held on _____

Examine-1**Examine -2****INDEX**

SI.NO	PROGRAM NAME	DATE	PAGE NUM	SIGNATURE
01(1.1)	INTRODUCTION TO PYTHON LIBRARIES FOR DATA SCIENCE	02-02-2026	1	
1.2	PANDAS	02-02-2026	5	
1.3	MATPLOT LIB	02-02-2026	7	
1.4	SCIKIT LEARN	09-02-2026	9	
02	PERFORM DATA EXPLORATION AND PRE PROCESSING IN PYTHON	14-02-2026	11	
03	IMPLEMENT REGULARIZED LINEAR REGRESSION	14-02-2026	13	
04	IMPLEMENT NAVIE BAYES CLASSIFIER	23-02-2026	14	
05	IMPLEMENT REGULARIZED LOGISTIC REGRESSION	02-03-2026	15	
06	BUILD MODULES USING DIFFERENT ENSEMBLING TECHNIQUES	09-03-2026	17	
07	BUILD MODELS USING DECESION TREES	09-03-2026	18	
08	BUILD THE MODELS USING DIFFERENT KERNAL FUNCTIONS	16-03-2026	19	
09	IMPLEMENT K-NEAREST NEIGHBOURS (KNN)CLASSIFICATION	30-03-2026	20	

10	K-MEAN CLUSTERING WITH PCA AND ELBOW METHOD	30-03-2026	21	
----	---	------------	----	--

INTRODUCTION TO PYTHON LIBRARIES FOR DATA SCIENCE

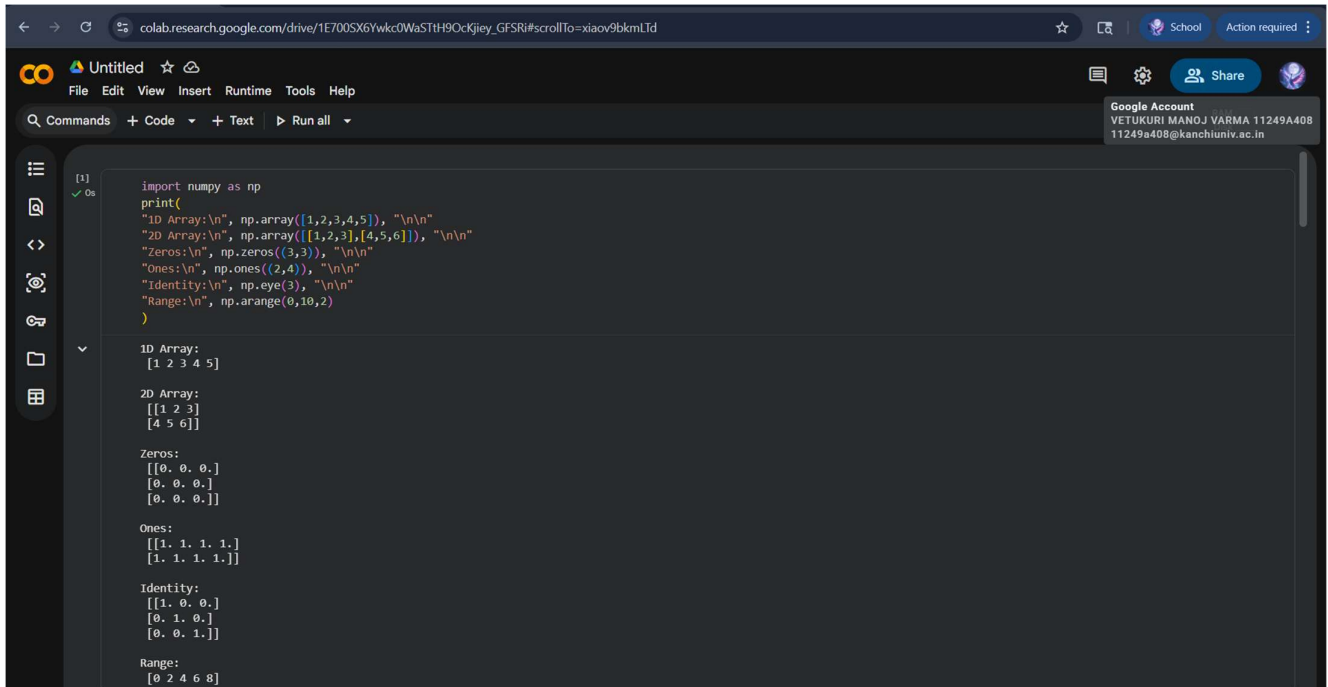
Exp-01(1.1)

Date-02-02-2026

Aim:

To learn basic Python data science libraries like NumPy and Pandas.

Program with output:



```
[1]
import numpy as np
print(
    "1D Array:\n", np.array([1,2,3,4,5]), "\n\n"
    "2D Array:\n", np.array([[1,2,3],[4,5,6]]), "\n\n"
    "Zeros:\n", np.zeros((3,3)), "\n\n"
    "Ones:\n", np.ones((2,4)), "\n\n"
    "Identity:\n", np.eye(3), "\n\n"
    "Range:\n", np.arange(0,10,2)
)
```

1D Array:
[1 2 3 4 5]

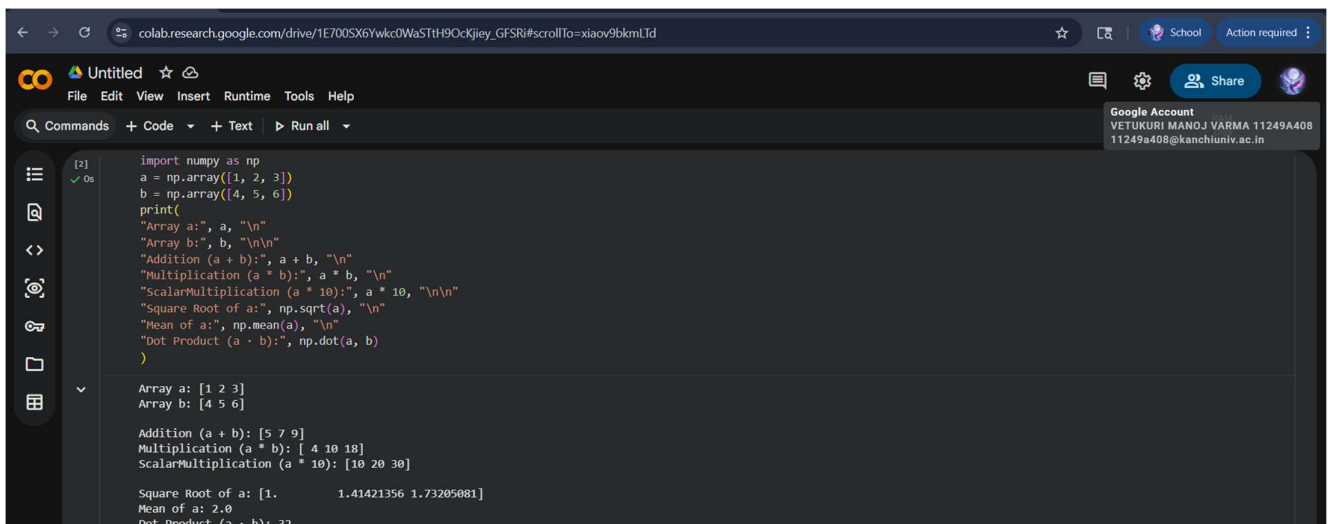
2D Array:
[[1 2 3]
[4 5 6]]

Zeros:
[[0. 0. 0.]
[0. 0. 0.]
[0. 0. 0.]]

Ones:
[[1. 1. 1. 1.]
[1. 1. 1. 1.]]

Identity:
[[1. 0. 0.]
[0. 1. 0.]
[0. 0. 1.]]

Range:
[0 2 4 6 8]



```
[2]
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(
    "Array a:", a, "\n"
    "Array b:", b, "\n\n"
    "Addition (a + b):", a + b, "\n"
    "Multiplication (a * b):", a * b, "\n"
    "ScalarMultiplication (a * 10):", a * 10, "\n\n"
    "Square Root of a:", np.sqrt(a), "\n"
    "Mean of a:", np.mean(a), "\n"
    "Dot Product (a · b):", np.dot(a, b)
)
```

Array a: [1 2 3]
Array b: [4 5 6]

Addition (a + b): [5 7 9]
Multiplication (a * b): [4 10 18]
ScalarMultiplication (a * 10): [10 20 30]

Square Root of a: [1. 1.41421356 1.73205081]
Mean of a: 2.0
Dot Product (a · b): 32

```
colab.research.google.com/drive/1E700SX6Ywkc0WaSTtH9OcKjey_GFSRI#scrollTo=xiaov9bkmlTd

Untitled
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

# 7. Indexing & Slicing
print("\nFirst element:", a[0])
print("Slice:", a[1:4])
print("2D Slice:\n", g[0:2, 1:3])

# 8. Boolean & Conditional
print("\nWhere > 50:\n", np.where(g > 50))
print("Any > 90:", np.any(g > 90))
print("All > 0:", np.all(g > 0))

# 9. Sorting & Searching
print("Unsorted a:", np.sort(a))
print("Index of max:", np.argmax(a))
print("Index of min:", np.argmin(a))
print("Unique elements:", np.unique([1, 2, 2, 3, 4, 4]))

# 10. Linear Algebra
x = np.array([1, 2], [3, 4])
y = np.array([5, 6], [7, 8])

print("\nDot Product:\n", np.dot(x, y))
print("Matrix Multiplication:\n", np.matmul(x, y))
print("Determinant:", np.linalg.det(x))
print("Inverse:\n", np.linalg.inv(x))

# 11. Random Functions
np.random.seed(1)
rand_sample = np.random.choice(a, size=3)
print("Random Choice:", rand_sample)

# 12. Stack & Concatenate
h1 = np.array([1, 2, 3])
h2 = np.array([4, 5, 6])

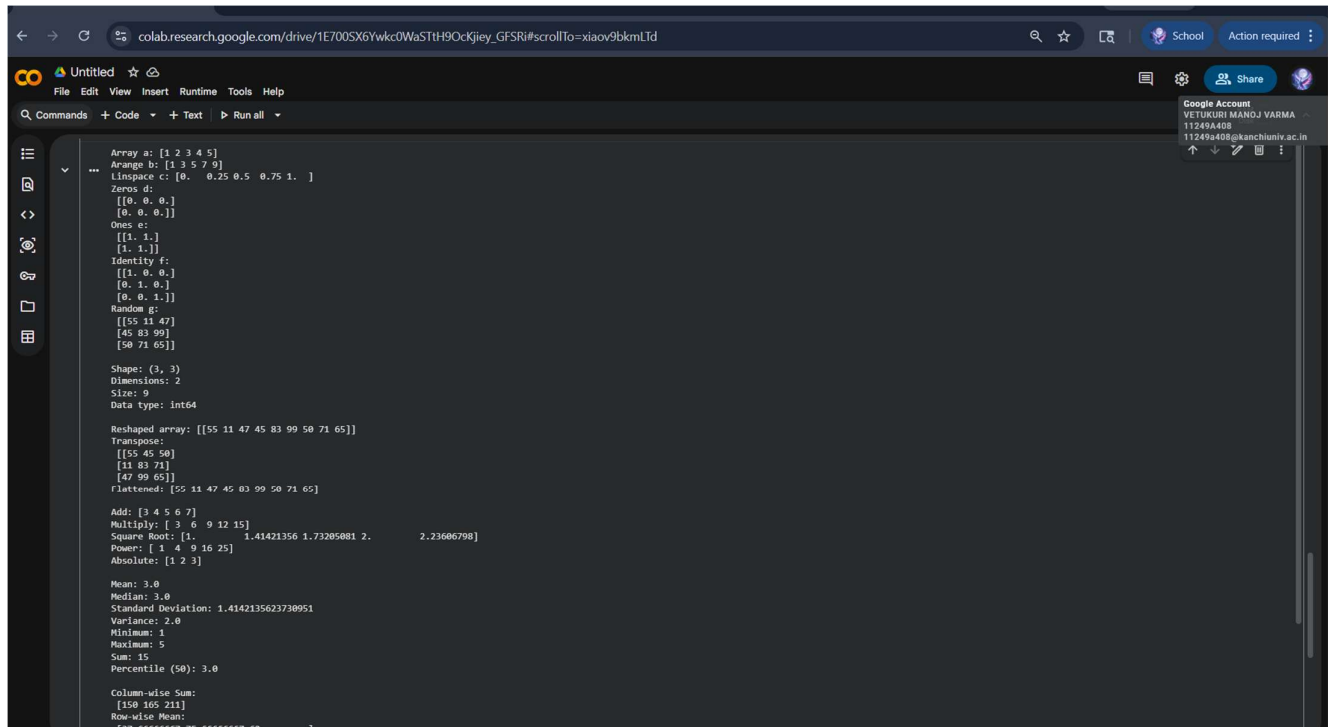
print("\nConcatenate:", np.concatenate((h1, h2)))
print("Vertical Stack:\n", np.vstack((h1, h2)))
print("Horizontal Stack:", np.hstack((h1, h2)))
```

OUTPUT-

```
colab.research.google.com/drive/1E700SX6Ywkc0WaSTtH9OcKjey_GFSRI#scrollTo=xiaov9bkmlTd

Untitled
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

Array a: [1 2 3 4 5]
Array b: [1 3 5 7 9]
Linspace c: [0.  0.25 0.5  0.75 1. ]
Zeros d:
[[0. 0. 0.]
 [0. 0. 0.]]
Ones e:
[[1. 1.]
 [1. 1.]]
Identity f:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
Random g:
[[55 11 47]
 [45 83 99]
 [50 71 65]]
Shape: (3, 3)
Dimensions: 2
Size: 9
Data type: int64
Reshaped array: [[55 11 47 45 83 99 50 71 65]]
Transpose:
[[55 45 50]
 [11 83 71]
 [47 99 65]]
Flattened: [55 11 47 45 83 99 50 71 65]
Add: [3 4 5 6 7]
Multiply: [ 3  6  9 12 15]
Square Root: [1.  1.41421356 1.73205081 2.  2.23606798]
Power: [ 1  4  9 16 25]
Absolute: [1 2 3]
Mean: 3.0
Median: 3.0
Standard Deviation: 1.4142135623730951
Variance: 2.0
Minimum: 1
Maximum: 5
Sum: 15
Percentile (50): 3.0
Column-wise Sum:
[150 165 211]
Row-wise Mean:
[17.66666667 75.66666667 67.66666667]
```



```
colab.research.google.com/drive/1E700SX6Ywkc0WaSTtH9OcKjiej_GFSRi#scrollTo=xiaov9bkmLTd

Untitled
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

Array a: [1 2 3 4 5]
Arange b: [1 3 5 7 9]
Linspace c: [0. 0.25 0.5 0.75 1. ]
Zeros d:
[[0. 0. 0.]
 [0. 0. 0.]]
Ones e:
[[1. 1.]
 [1. 1.]]
Identity f:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
Random g:
[[55 11 47]
 [45 83 99]
 [58 71 65]]

Shape: (3, 3)
Dimensions: 2
Size: 9
Data type: int64

Reshaped array: [[55 11 47 45 83 99 58 71 65]]
Transpose:
[[55 45 58]
 [11 83 71]
 [47 99 65]]
Flattened: [55 11 47 45 83 99 58 71 65]

Add: [3 4 5 6 7]
Multiply: [ 3 6 9 12 15]
Square Root: [1. 1.41421356 1.73205081 2. 2.23606798]
Power: [ 1 4 9 16 25]
Absolute: [1 2 3]

Mean: 3.0
Median: 3.0
Standard Deviation: 1.4142135623738951
Variance: 2.0
Minimum: 1
Maximum: 5
Sum: 15
Percentile (50): 3.0

Column-wise Sum:
[158 165 211]
Row-wise Mean:
[37.66666667 75.66666667 67.33333333]
```

Result:

Thus, it is implemented *Introduction to Python Libraries for Data Science* successfully.

PANDAS

EXP-1.2 DATE-02-02-2026

Aim:

To study the Pandas library for data handling and analysis.

Program with output:

```
colab.research.google.com/drive/1E700SX6Ywkc0WaSTtH9OcKjey_GFSRi#scrollTo=d3GDxLTr6nT

Untitled
File Edit View Insert Runtime Tools Help
Commands + Code + Text + Run all

import pandas as pd
data = {
    'name': ['alice', 'bob', 'claire'],
    'age': [25, 30, 35],
    'city': ['new york', 'paris', 'london']
}
df = pd.DataFrame(data)
print(df.info())
print(df.info())
print(df.describe())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  --
0   name    3 non-null          object
1   age     3 non-null          int64
2   city    3 non-null          object
dtypes: int64(1), object(2)
memory usage: 284.0+ bytes
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  --
0   name    3 non-null          object
1   age     3 non-null          int64
2   city    3 non-null          object
dtypes: int64(1), object(2)
memory usage: 284.0+ bytes
None
count    age
mean    30.0
std      5.0
min     25.0
25%     27.5
50%     30.0
75%     32.5
max     35.0
```

Program with output:

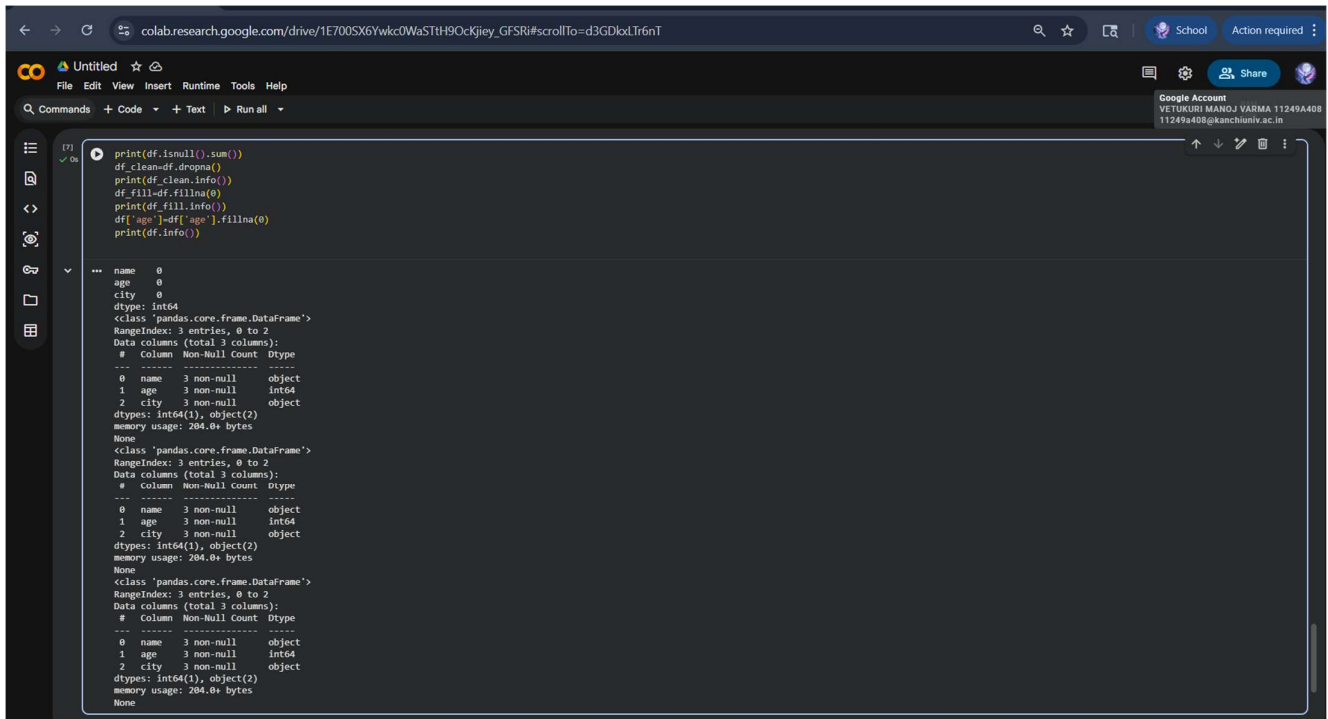
```
colab.research.google.com/drive/1E700SX6Ywkc0WaSTtH9OcKjey_GFSRi#scrollTo=d3GDxLTr6nT

Untitled
File Edit View Insert Runtime Tools Help
Commands + Code + Text + Run all

import pandas as pd
data = {
    'name': ['alice', 'bob', 'claire'],
    'age': [25, 30, 35],
    'city': ['new york', 'paris', 'london']
}
df = pd.DataFrame(data)
print(df.info())
print(df.info())
print(df.describe())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  --
0   name    3 non-null          object
1   age     3 non-null          int64
2   city    3 non-null          object
dtypes: int64(1), object(2)
memory usage: 284.0+ bytes
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  --
0   name    3 non-null          object
1   age     3 non-null          int64
2   city    3 non-null          object
dtypes: int64(1), object(2)
memory usage: 284.0+ bytes
None
count    age
mean    30.0
std      5.0
min     25.0
25%     27.5
50%     30.0
75%     32.5
max     35.0
```

Program with output:



The screenshot shows a Google Colab notebook interface. The top bar includes the Google Colab logo, the name 'Untitled', and various icons for file management, search, and sharing. Below the top bar is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. The main area is divided into a code editor on the left and an output viewer on the right. The code editor contains the following Python code:

```
print(df.isnull().sum())
df_clean=df.dropna()
print(df_clean.info())
df_fill=df.fillna(0)
print(df_fill.info())
df["age"]-=df["age"].fillna(0)
print(df.info())
```

The output viewer displays the results of these operations. It shows the initial DataFrame with 3 entries and 3 columns (name, age, city). The first operation, `df.isnull().sum()`, returns 0 for all columns. The second operation, `df_clean=df.dropna()`, returns a DataFrame with 3 entries and 3 columns. The third operation, `df_fill=df.fillna(0)`, returns a DataFrame with 3 entries and 3 columns. The final operation, `df["age"]-=df["age"].fillna(0)`, returns a DataFrame with 3 entries and 3 columns. The output also shows the memory usage of the DataFrame, which is 204.0+ bytes.

Result:

Thus, it is implemented *Pandas* successfully.

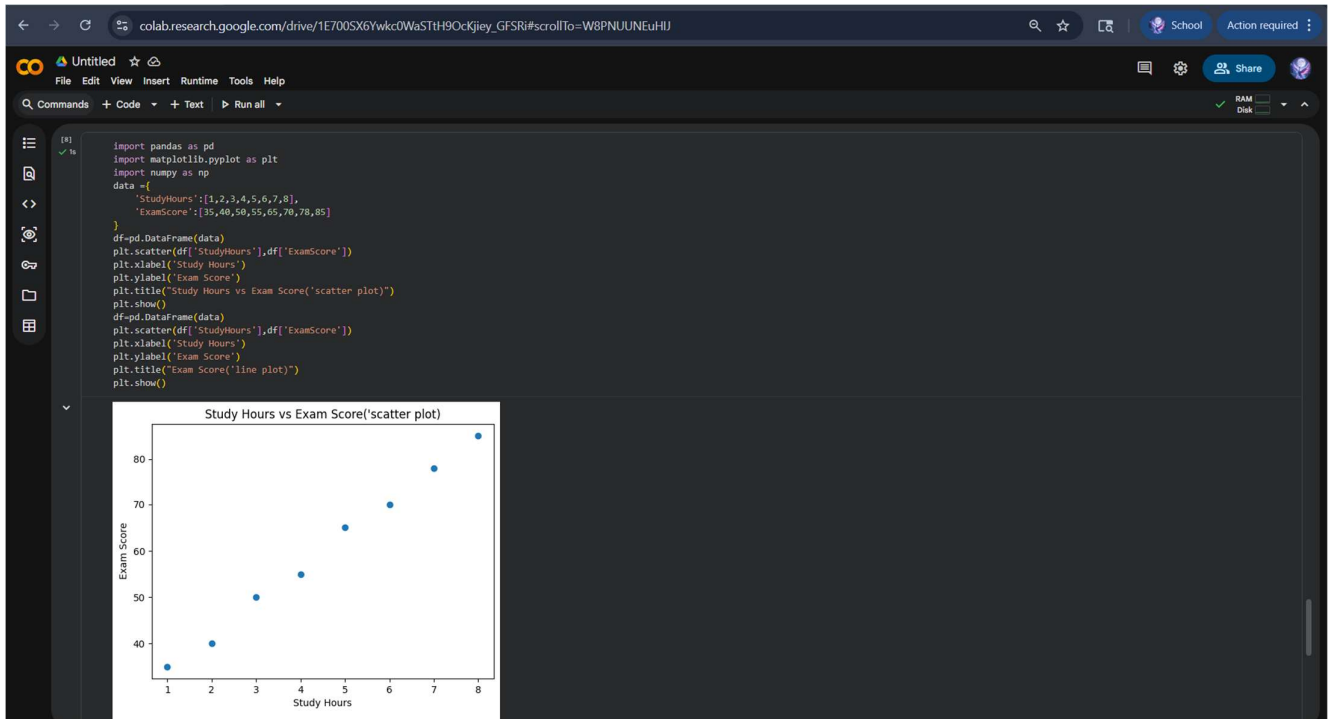
MATPLOTLIB

EXP-1.3 DATE-02-02-2026

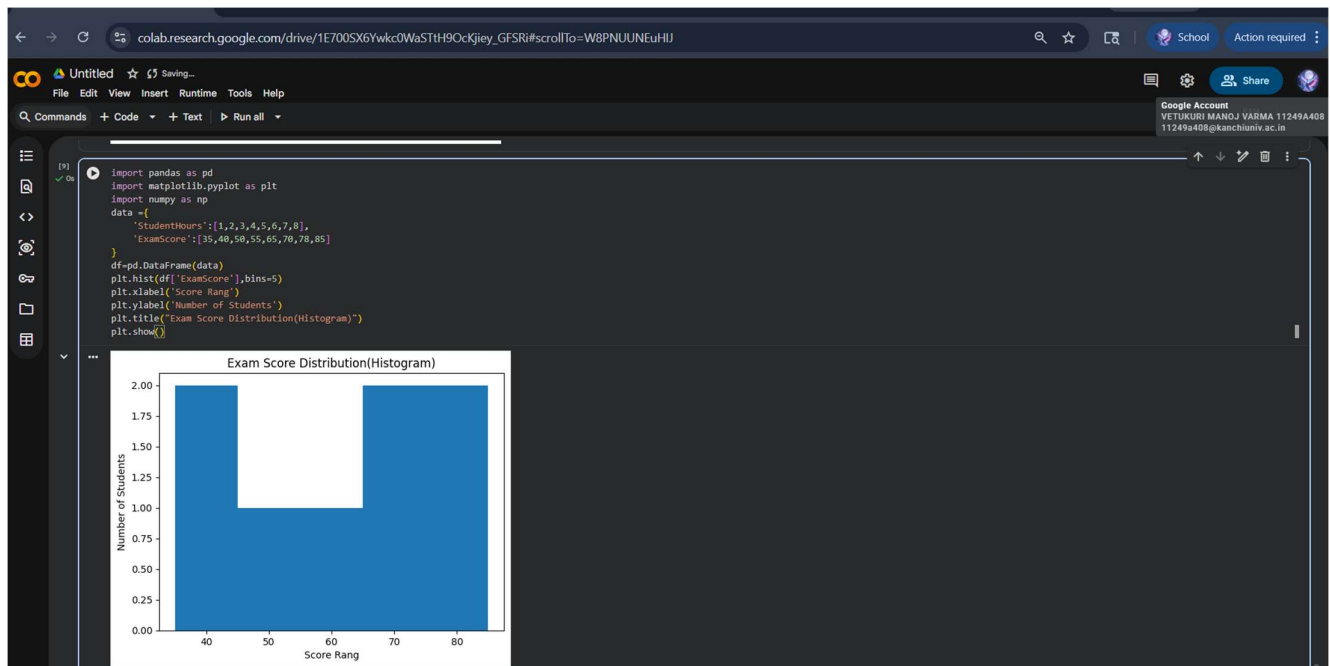
Aim:

To study the Matplotlib library for data visualization.

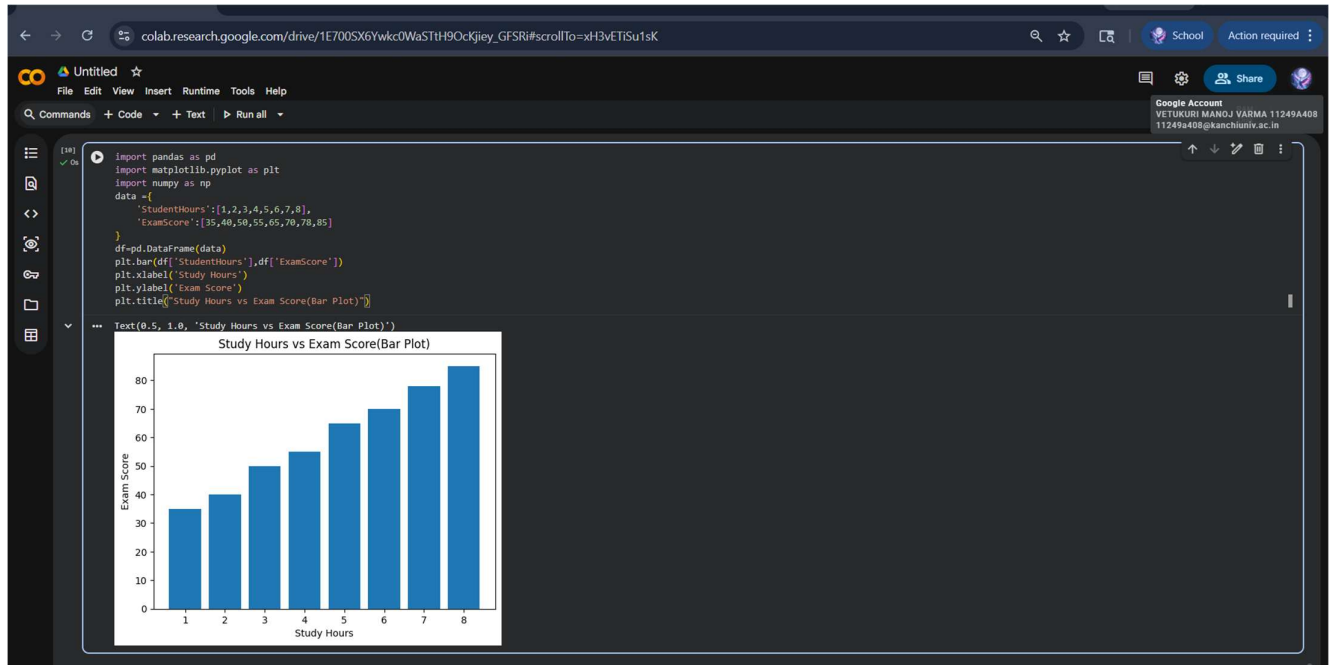
PROGRAM WITH OUTPUT-



PROGRAM WITH OUTPUT-



PROGRAM WITH OUTPUT-



Result:

Thus, it is implemented *Matplotlib* successfully.

SCIKIT LEARN

EXP N-1.4 DATE-09-02-2026

Aim:

To study the scikit-learn library for machine learning.

PROGRAM WITH OUTPUT-

```
colab.research.google.com/drive/1E700SX6Ywkc0WaSTtH9OcKjley_GFSRi#scrollTo=iPIEKs18vFKK

Untitled
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

data = {
    'StudyHours': ['S1', 'S2', 'S3', 'S4', 'S5', 'S6', 'S7', 'S8', 'S9', 'S10'],
    'Class_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
    'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}

df=pd.DataFrame(data)
print(df)
```

	StudyHours	Class_Attended	Internal_Marks
0	S1	30	35
1	S2	35	38
2	S3	40	42
3	S4	45	46
4	S5	50	50
5	S6	55	55
6	S7	60	60
7	S8	65	65
8	S9	70	70
9	S10	75	75

PROGRAM WITH OUTPUT-

```
colab.research.google.com/drive/1E700SX6Ywkc0WaSTtH9OcKjley_GFSRi#scrollTo=REPol2HwMqA

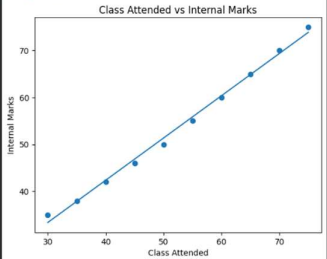
Untitled
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

data = {
    'StudyHours': ['S1', 'S2', 'S3', 'S4', 'S5', 'S6', 'S7', 'S8', 'S9', 'S10'],
    'Class_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
    'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}

df=pd.DataFrame(data)
X=df[['Class_Attended']]
y=df['Internal_Marks']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=7)
model=LinearRegression()
model.fit(X_train,y_train)
predictions=model.predict(X_test)
mse=mean_squared_error(y_test,predictions)
print("MSE:",mse)
print("Marks per Class:",model.coef_[0])
print("Base Marks:",model.intercept_)
plt.scatter(X,y)
plt.plot(X,model.predict(X))
plt.xlabel('Class Attended')
plt.ylabel('Internal Marks')
plt.title('Class Attended vs Internal Marks')
plt.show()
```

MSE: 0.578125
Marks per Class: 0.9
Base Marks: 0.975



Result:

Thus, it is implemented *Scikit-learn* successfully.

PERFORM DATA EXPLORATION AND PREPROCESSING IN PYTHON

EXP-02

DATE-14-02-2026

Aim:

To perform data exploration and preprocessing using Python.

PROGRAM :

```
colab.research.google.com/drive/1E700SX6Ywkc0WastU1H9Ockjey_GFSRI#scrollTo=w6BRC_iMw8G3

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

data = {
    'country': ['India', 'USA', 'India', 'USA', 'UK', 'India'],
    'Age': [22, 25, np.nan, 30, 28, 35],
    'salary': [400000, 60000, 50000, np.nan, 72000, 50000],
    'purchased': ['no', 'yes', 'yes', 'no', 'yes', 'yes']
}

df = pd.DataFrame(data)
print("---ORIGINAL RAW DATA---")
print(df)
print("\n")
print("---MISSING VALUE COUNT---")
print(df.isnull().sum())
print("\n")
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['salary'] = df['salary'].fillna(df['salary'].mean())
print("---DATA AFTER CLEANING---")
print(df)
print("\n")
df_encoded = pd.get_dummies(df, columns=['country'])
df_encoded['purchased'] = df_encoded['purchased'].map({'yes': 1, 'no': 0})
print("---DATA AFTER ENCODING(All Number Now---")
print(df_encoded)
print("\n")
df_cleaned = df_encoded.copy()
df_cleaned['Age'] = df_cleaned['Age'].fillna(df_cleaned['Age'].mean())
df_cleaned['salary'] = df_cleaned['salary'].fillna(df_cleaned['salary'].mean())
df_encoded = pd.get_dummies(df_cleaned, columns=['country'])
df_encoded['purchased'] = df_encoded['purchased'].map({'yes': 1, 'no': 0})

x = df_encoded.drop(['purchased', 'country'], axis=1)
y = df_encoded['purchased']

scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
print("---FINAL PROCESSED DATA(Ready for AI---")
print(pd.DataFrame(x_scaled, columns=x.columns).head())
print("\n")
```

Output:

```
colab.research.google.com/drive/1E700SX6Ywkc0WastU1H9Ockjey_GFSRI#scrollTo=w6BRC_iMw8G3

---ORIGINAL RAW DATA---
   country  Age  salary purchased
0  India    22.0  400000.0       no
1  USA     25.0   60000.0       yes
2  India    NaN   50000.0       yes
3  USA     30.0    NaN       no
4  UK      28.0   72000.0       yes
5  India    35.0   50000.0       yes

---MISSING VALUE COUNT---
country    0
Age         1
salary      1
purchased   0
dtype: int64

---DATA AFTER CLEANING---
   country  Age  salary purchased
0  India    22.0  400000.0       no
1  USA     25.0   60000.0       yes
2  India    28.0   50000.0       yes
3  USA     30.0  120000.0       no
4  UK      28.0   72000.0       yes
5  India    35.0   50000.0       yes

---DATA AFTER ENCODING(All Number Now---
   Age  salary purchased  country_UK  country_USA  country_India  Purchased
0  22.0  400000.0       no         False         False          True         0
1  25.0   60000.0       yes         False          True         False         1
2  28.0   50000.0       yes         False         False          True         1
3  30.0  120000.0       no         False          True         False         0
4  28.0   72000.0       yes         True          False         False         1
5  35.0   50000.0       yes         False         False          True         1

---FINAL PROCESSED DATA(Ready for AI---
   Age  salary  country_UK  country_USA  country_India
0 -1.484615  2.187959  -0.447214  -0.707107         1.0
1 -0.742187 -0.546990 -0.447214  1.414214        -1.0
2  0.000000 -0.627429 -0.447214  -0.707107         1.0
3  0.494272  0.000000 -0.447214  1.414214        -1.0
4  0.000000 -0.459402  2.236068  -0.707107        -1.0
```

Result:

The dataset was explored, cleaned, and pre processed successfully, making it ready for further analysis or model building.

IMPLEMENT REGULARIZED LINEAR REGRESSION

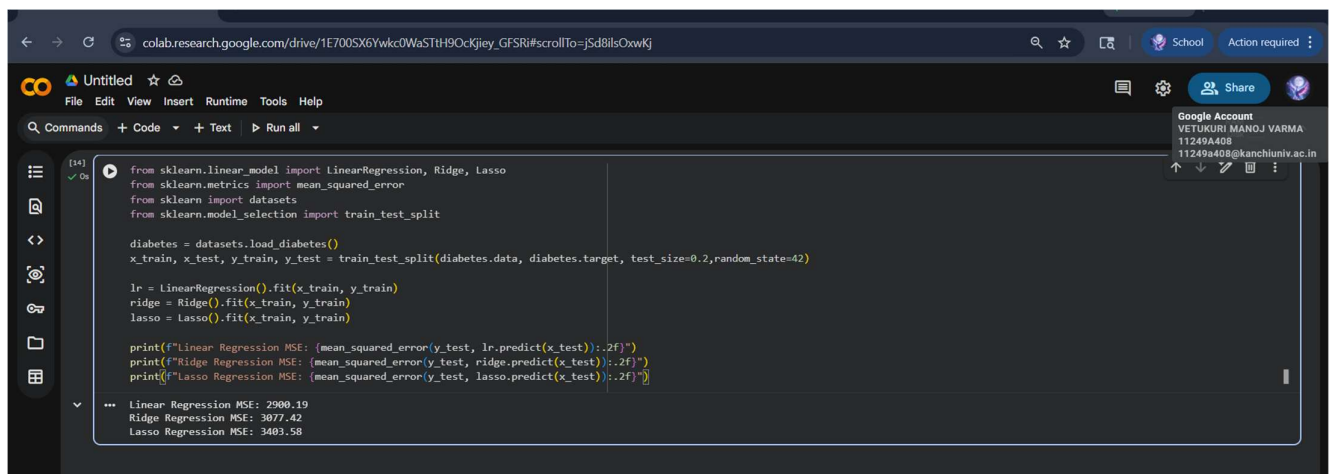
EXP-03

DATE-14-02-2026

Aim

To implement Regularized Linear Regression using Python.

PROGRAM WITH OUTPUT



```
[14] ✓ Os
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
from sklearn import datasets
from sklearn.model_selection import train_test_split

diabetes = datasets.load_diabetes()
x_train, x_test, y_train, y_test = train_test_split(diabetes.data, diabetes.target, test_size=0.2, random_state=42)

lr = LinearRegression().fit(x_train, y_train)
ridge = Ridge().fit(x_train, y_train)
lasso = Lasso().fit(x_train, y_train)

print(f"Linear Regression MSE: {mean_squared_error(y_test, lr.predict(x_test)):.2f}")
print(f"Ridge Regression MSE: {mean_squared_error(y_test, ridge.predict(x_test)):.2f}")
print(f"Lasso Regression MSE: {mean_squared_error(y_test, lasso.predict(x_test)):.2f}")

...
Linear Regression MSE: 2900.19
Ridge Regression MSE: 3077.42
Lasso Regression MSE: 3403.58
```

Result

The regularized linear regression model was successfully implemented using Python. The model reduced overfitting and improved prediction performance by applying regularization techniques such as Ridge or Lasso regression.

IMPLEMENT NAVIE BAYES CLASSIFIER

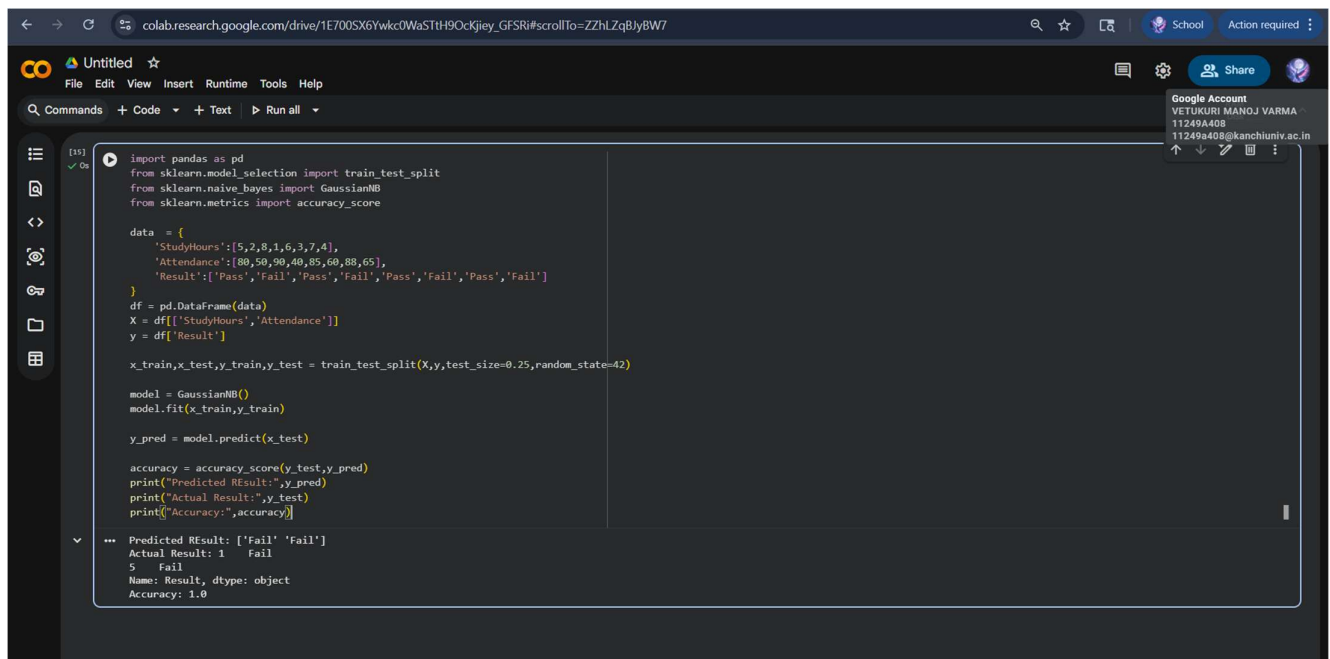
EXP-04

DATE-23-02-2026

Aim

To implement a Naive Bayes classifier using Python for classification of data.

PROGRAM WITH OUTPUT



```
[15]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

data = {
    'StudyHours': [5, 2, 8, 1, 6, 3, 7, 4],
    'Attendance': [80, 50, 90, 40, 85, 60, 88, 65],
    'Result': ['Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail']
}

df = pd.DataFrame(data)
X = df[['StudyHours', 'Attendance']]
y = df['Result']

x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

model = GaussianNB()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)
print("Predicted Result:", y_pred)
print("Actual Result:", y_test)
print("Accuracy:", accuracy)
```

... Predicted Result: ['Fail' 'Fail']
Actual Result: 1 Fail
5 Fail
Name: Result, dtype: object
Accuracy: 1.0

Result

The Naive Bayes classifier was successfully implemented using Python. The model was trained and used to classify the data effectively, providing accurate predictions based on probability.

IMPLEMENT REGULARIZED LOGISTIC REGRESSION

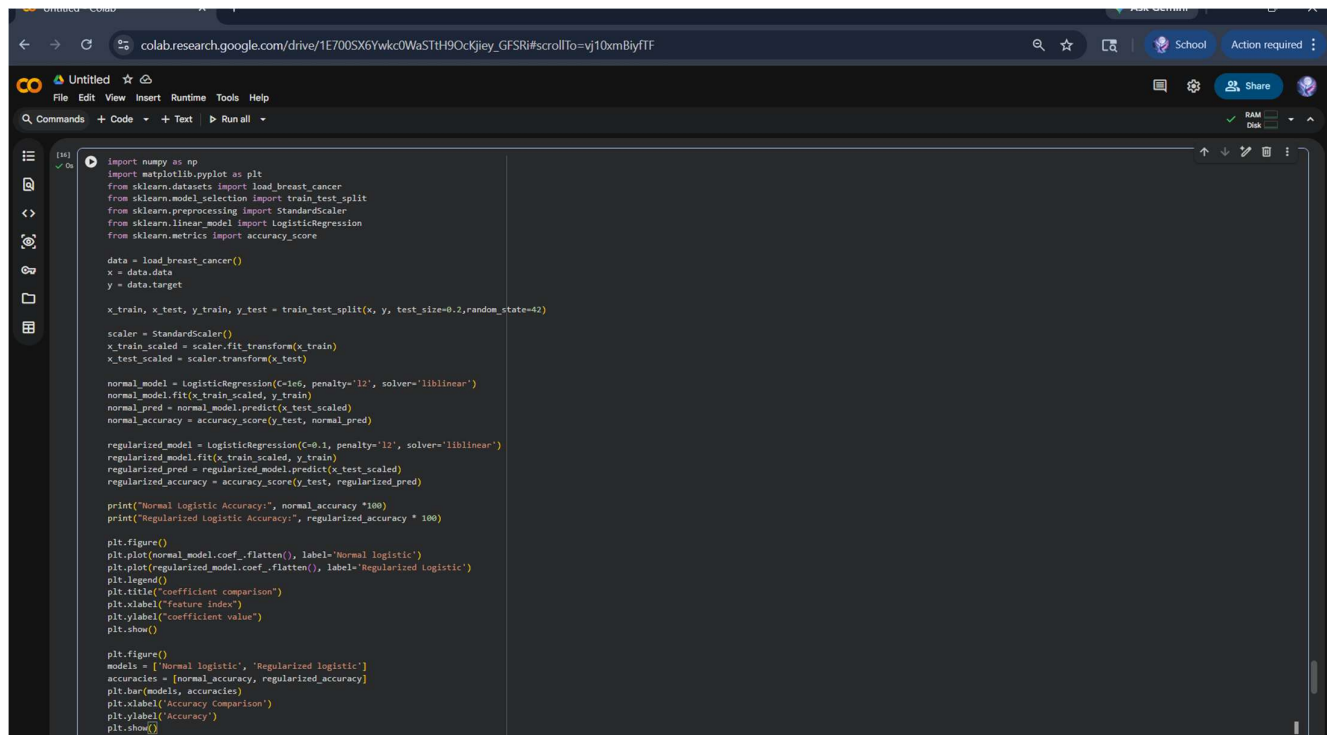
EXP-05

DATE-02-03-2026

Aim:

To implement Regularized Logistic Regression using Python.

Program:



```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
x = data.data
y = data.target

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')
normal_model.fit(x_train_scaled, y_train)
normal_pred = normal_model.predict(x_test_scaled)
normal_accuracy = accuracy_score(y_test, normal_pred)

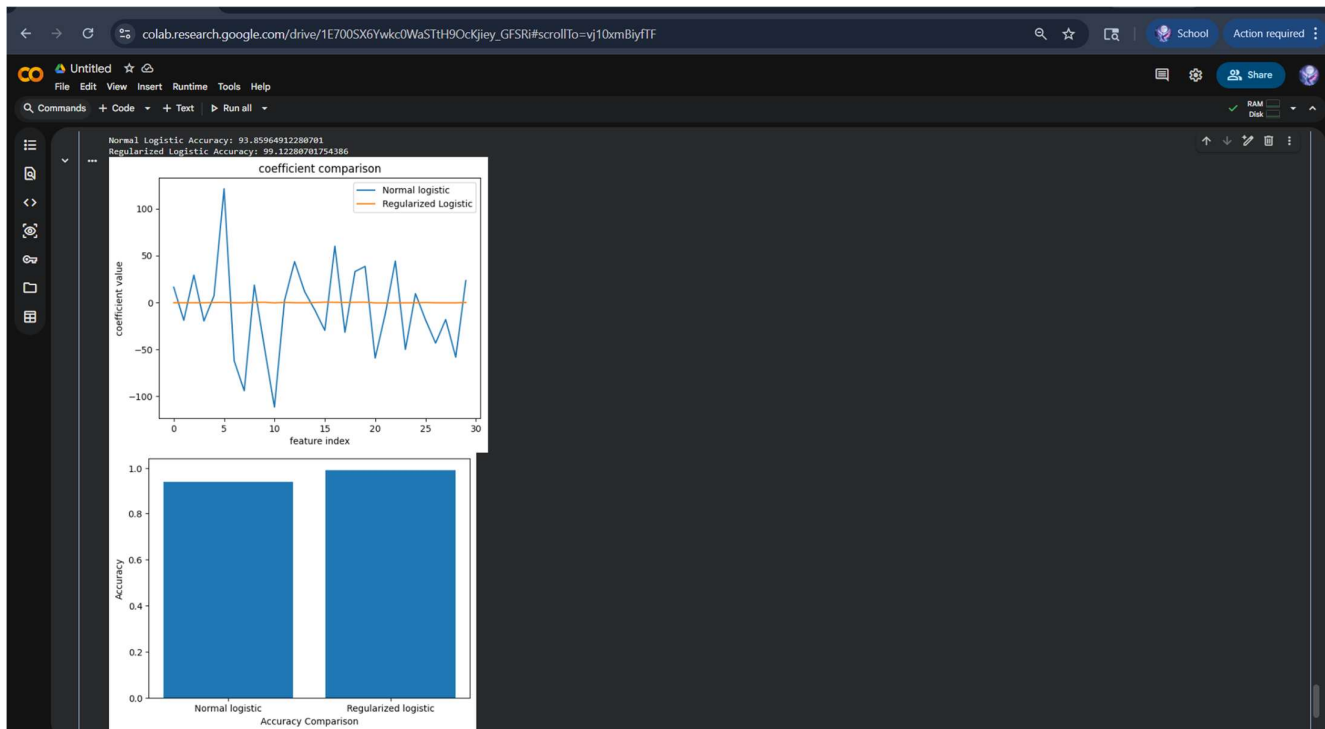
regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
regularized_model.fit(x_train_scaled, y_train)
regularized_pred = regularized_model.predict(x_test_scaled)
regularized_accuracy = accuracy_score(y_test, regularized_pred)

print("Normal Logistic Accuracy:", normal_accuracy * 100)
print("Regularized Logistic Accuracy:", regularized_accuracy * 100)

plt.figure()
plt.plot(normal_model.coef_.flatten(), label='Normal logistic')
plt.plot(regularized_model.coef_.flatten(), label='Regularized logistic')
plt.legend()
plt.title('Coefficient comparison')
plt.xlabel('feature index')
plt.ylabel('coefficient value')
plt.show()

plt.figure()
models = ['Normal logistic', 'Regularized logistic']
accuracies = [normal_accuracy, regularized_accuracy]
plt.bar(models, accuracies)
plt.xlabel('Accuracy Comparison')
plt.ylabel('Accuracy')
plt.show()
```

Output:



Result

The regularized logistic regression model was successfully implemented using Python. Regularization helped reduce overfitting and improved the model's classification performance and generalization on the dataset.

BUILD MODULES USING DIFFERENT ENSEMBLING TECHNIQUES

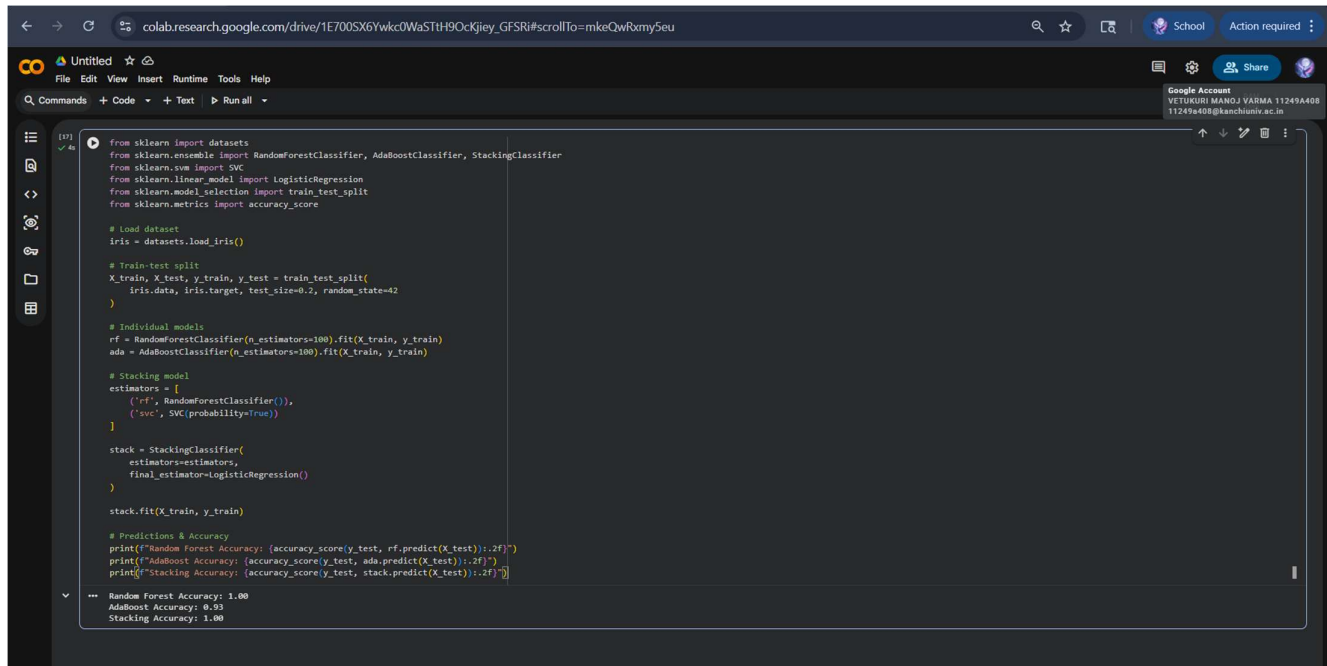
EXP-06

DATE-09-03-2026

Aim:

To build models using different ensemble techniques in Python.

PROGRAM WITH OUTPUT



```
[17]: from sklearn import datasets
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
iris = datasets.load_iris()

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

# Individual models
rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)

# Stacking model
estimators = [
    ("rf", RandomForestClassifier()),
    ("svc", SVC(probability=True))
]

stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
)

stack.fit(X_train, y_train)

# Predictions & Accuracy
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00

Result

The ensemble models were successfully built using techniques such as Bagging, Boosting, and Random Forest. These methods improved prediction accuracy and reduced overfitting compared to individual models.

BUILD MODELS USING DECISION TREES

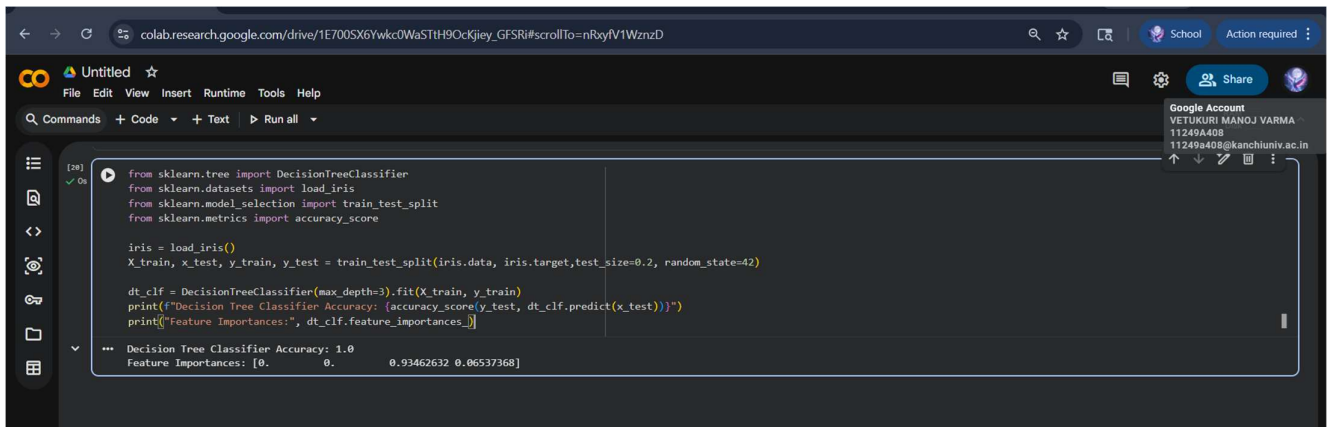
EXP-7

DATE-09-03-2026

Aim:

To build a classification model using Decision Trees in Python.

PROGRAM WITH OUTPUT



The screenshot shows a Google Colab notebook interface. The browser address bar displays a Google Drive link. The notebook has a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. The code editor contains the following Python code:

```
[28] ✓ 0s
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris = load_iris()
X_train, x_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.2, random_state=42)

dt_clf = DecisionTreeClassifier(max_depth=3).fit(X_train, y_train)
print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test, dt_clf.predict(x_test))}")
print("Feature Importances:", dt_clf.feature_importances_)

*** Decision Tree Classifier Accuracy: 1.0
Feature Importances: [0. 0. 0.93462632 0.06537368]
```

On the right side, a 'Share' button and a 'Google Account' dropdown menu are visible. The account dropdown shows the name 'VETUKURI MANOJ VARMA' and the email '112499408@kanchiuniv.ac.in'.

Result:

The Decision Tree model was successfully implemented using Python. The model learned decision rules from the dataset and provided accurate predictions for classification tasks.

BUILD THE MODELS USING DIFFERENT KERNAL FUNCTIONS

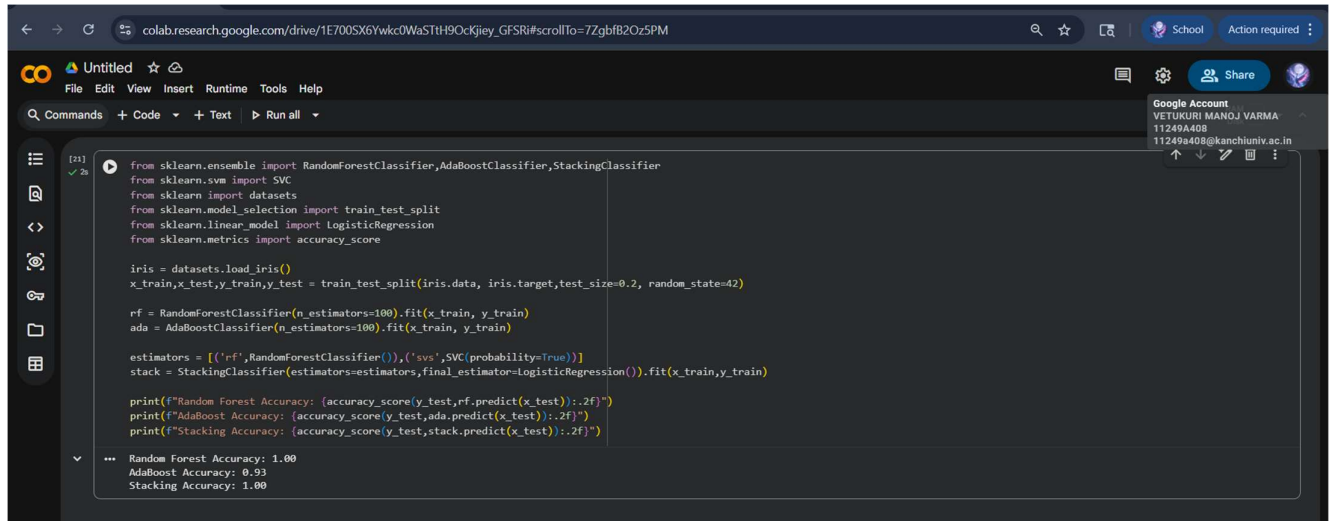
EXP-08

DATE-16-03-2026

Aim

To build models using different kernel functions in Python.

PROGRAM WITH OUTPUT



```
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

iris = datasets.load_iris()
x_train, x_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.2, random_state=42)

rf = RandomForestClassifier(n_estimators=100).fit(x_train, y_train)
ada = AdaBoostClassifier(n_estimators=100).fit(x_train, y_train)

estimators = [('rf', RandomForestClassifier()), ('svm', SVC(probability=True))]
stack = StackingClassifier(estimators=estimators, final_estimator=LogisticRegression()).fit(x_train, y_train)

print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(x_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(x_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(x_test)):.2f}")
```

Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00

Result

The model was successfully implemented using kernel-based methods (such as SVM). Different kernel functions like linear, polynomial, and RBF were used, and they improved classification performance based on the dataset structure.

IMPLEMENT K-NEAREST NEIGHBOURS (KNN) CLASSIFICATION

EXP-09

DATE-30-3-2026

Aim:

To implement the K-Nearest Neighbours (KNN) classification algorithm using Python.

PROGRAM:

```
colab.research.google.com/drive/1E700SX6Ywkc0WaStH9OcKjey_GFSRI#scrollTo=DkIbbVJH0Wai

Untitled
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

# Step 1: Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Create a small example dataset (similar to mobile dataset)
data = {
    'battery_power': [842, 1821, 563, 615, 1821, 1859, 1954, 1445],
    'ram': [2549, 2631, 2683, 2769, 1411, 1867, 275, 296],
    'px_height': [20, 905, 1263, 1216, 1208, 1004, 381, 512],
    'px_width': [756, 1988, 1716, 1786, 1212, 1654, 1818, 1140],
    'price_range': [1, 2, 2, 2, 1, 1, 0, 0] # Target (0-3 in real dataset)
}

df = pd.DataFrame(data)

# Step 3: Features and target
X = df.drop(columns=['price_range'])
y = df['price_range']

# Step 4: Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.5, random_state=42, stratify=y
)

# Step 5: Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 6: Create KNN model
knn = KNeighborsClassifier(n_neighbors=3)

# Step 7: Train model
knn.fit(X_train, y_train)

# Step 8: Prediction
y_pred = knn.predict(X_test)

# Step 9: Evaluation
print("Actual:", list(y_test))
print("Predicted:", list(y_pred))

print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Output:

```
colab.research.google.com/drive/1E700SX6Ywkc0WaStH9OcKjey_GFSRI#scrollTo=DkIbbVJH0Wai

Untitled
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

Actual: [2, 0, 1, 1]
Predicted: [np.int64(2), np.int64(0), np.int64(0), np.int64(0)]

Accuracy: 0.5

Confusion Matrix:
[[1 0 0]
 [2 0 0]
 [0 0 1]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.33	1.00	0.50	1
1	0.00	0.00	0.00	2
2	1.00	1.00	1.00	1
accuracy				4
macro avg	0.44	0.67	0.50	4
weighted avg	0.33	0.50	0.38	4

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control.
warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control.
warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control.
warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Result

The KNN classification model was successfully implemented using Python. The model classified data based on the nearest neighbors and produced effective and accurate predictions for the given dataset.

K-MEAN CLUSTERING WITH PCA AND ELBOW METHOD

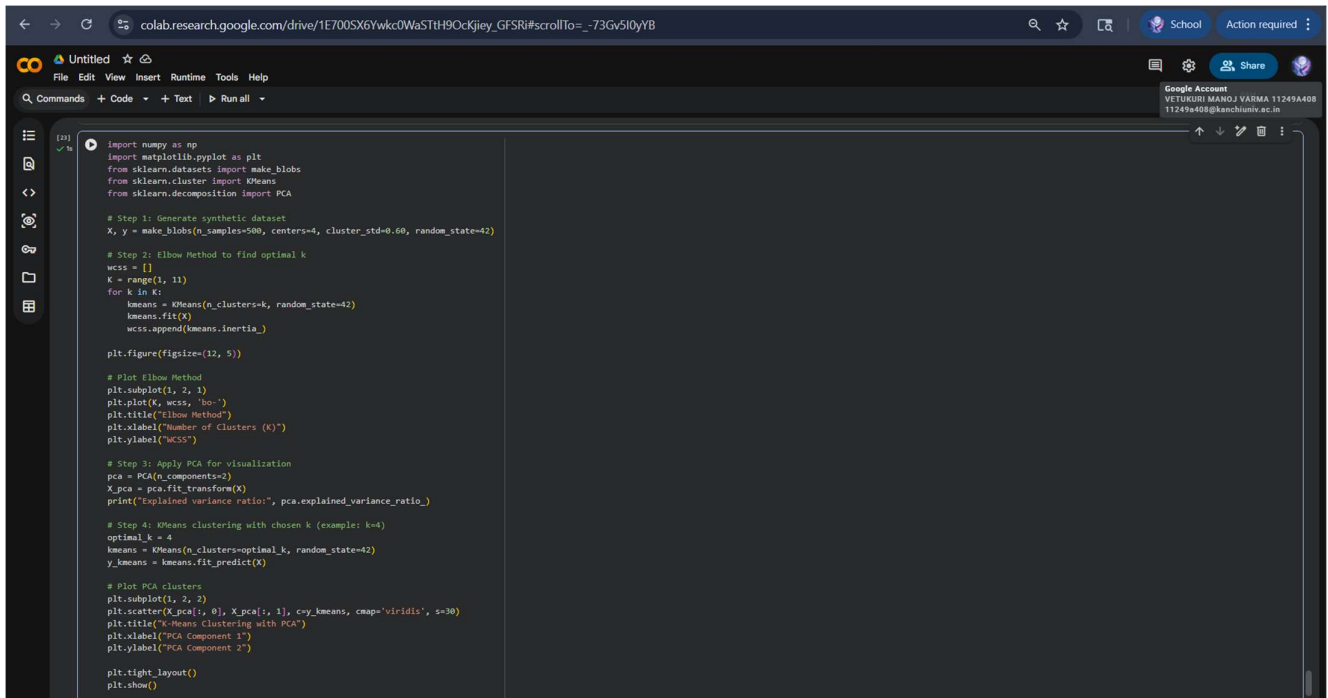
EXP-10

DATE-30-03-2026

Aim

To perform K-Means clustering using Python along with PCA and the Elbow method.

Program:



```
[21] import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# Step 1: Generate synthetic dataset
X, y = make_blobs(n_samples=500, centers=4, cluster_std=0.60, random_state=42)

# Step 2: Elbow Method to find optimal k
wcss = []
K = range(1, 11)
for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(12, 5))

# Plot Elbow Method
plt.subplot(1, 2, 1)
plt.plot(K, wcss, 'bo-')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("WCSS")

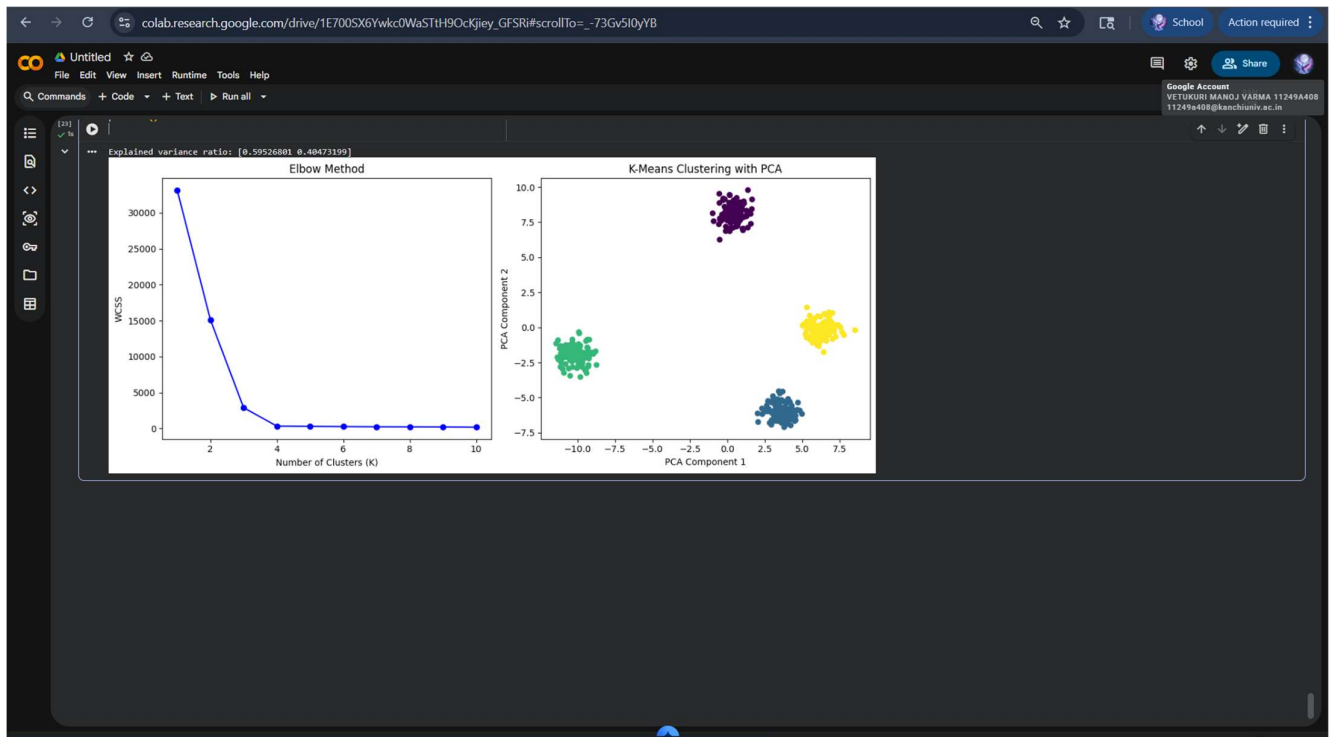
# Step 3: Apply PCA for visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
print("Explained variance ratio:", pca.explained_variance_ratio_)

# Step 4: KMeans clustering with chosen k (example: k=4)
optimal_k = 4
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
y_kmeans = kmeans.fit_predict(X)

# Plot PCA clusters
plt.subplot(1, 2, 2)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_kmeans, cmap='viridis', s=30)
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")

plt.tight_layout()
plt.show()
```

OUTPUT:



Result

K-Means clustering was successfully implemented. The optimal number of clusters was identified using the Elbow method, and PCA was used to reduce dimensionality for better visualization. The data was grouped into meaningful clusters effectively.